CAP-2459-IT

ChainCert:

A Permissioned Blockchain-Enabled Credential Verification and Management Platform

November 26, 2025

Demo Kit/Walkthrough

About

ChainCert is a decentralized credential management system built on Hyperledger Fabric blockchain technology. It provides a secure, tamper-proof platform for issuing, managing, and verifying academic and professional credentials.

The system enables:

- Issuers (accredited organizations like universities) to create credential templates and issue verifiable credentials to users
- Users (credential holders) to receive, store, and share their credentials securely
- Verifiers (employers, institutions) to verify the authenticity of credentials via blockchain queries (currently via CLI; web dashboard planned)
- Admins to manage issuer registrations and maintain system integrity

ChainCert leverages blockchain's immutability to ensure credentials cannot be forged or tampered with, while CouchDB provides off-chain storage for efficient data retrieval.

Important Notes

- The system requires a running Hyperledger Fabric network with the ChainCert chaincode deployed
- Each issuer organization operates its own peer node on the blockchain network
- User passwords are hashed using bcrypt for security
- All credential-related transactions are recorded immutably on the blockchain ledger
- The system uses dual CouchDB instances:
    - Port 5985: User authentication database
    - Port 9084: Issuer credential storage (per-organization)

- For the demo:
    - The DLSU organization node has been pre-configured and joined to the network for time efficiency. The node creation process (CA setup, peer deployment, channel joining, chaincode installation) was completed prior to the demonstration.
- Accredited Organizations Database:
    - In real life production, this would connect to a government API (e.g., CHED, PRC databases) to validate legitimate organizations.
    - For our capstone/demo, mock data is used as we do not have a client, with De La Salle University as an accredited organization.

Accounts

Admin Account

Field	**Value**
Email	admin@email.com
Password	pw123
Role	admin

Issuer Account (DLSU) – Pre-Deployed Node for Demo

Field	**Value**
Email	dlsu@email.com
Password	pw123

Role	issuer
Organization	De La Salle University
Status	Approved

User Account (Holder)

Field	**Value**
Email	kyle_justin_tan@dlsu.edu.ph
Password	pw123
Role	user
Full Name	Kyle Justin Tan
Holder ID	4c7e8e4c20da8359fcdd204e5402d34f

Scenarios

Scenario #1: Organization Registration – Non-Accredited Organization (Failed Registration)

Context: "Let's say there's this organization calling itself 'Fake University Inc.' They want to get into the ChainCert network and start issuing their own credentials — maybe fake diplomas, maybe worthless certificates. Whatever makes them money. "Fake University Inc." attempts to register as an issuer on ChainCert. However, this organization is NOT in the accredited organizations database

Step 1: The organization navigates to the ChainCert registration page and fills out the issuer registration form:

Field	**Value**
Organization Name	Fake University Inc.
Email	admin@fakeuniversity.com
Password	password123
Supporting Documents	(uploaded files)

Step 2: The organization clicks "Register".

Step 3: The system checks the `accredited_organizations` database (in production, this would be connected to CHED/government API).

Step 4: Since "Fake University Inc." is NOT found in the accredited organizations database, the registration fails.

Step 5: The system displays an error message:

Registration Failed
"Fake University Inc." is not a recognized accredited organization. Only institutions verified through CHED or other accrediting bodies may register as issuers on ChainCert.

Why This Matters:

- Prevents spam and fraudulent organizations from joining the network
- Ensures only legitimate institutions can issue credentials

- "Garbage in, garbage out" — if malicious orgs enter, malicious credentials get issued
- Protects the integrity and trustworthiness of the entire ChainCert ecosystem

---

Scenario #2: Organization Registration – Accredited Organization (Successful Registration, Pending Approval)

Context: Now let's contrast that with a REAL, legitimate organization (part of accredited_organizations database) De La Salle University (DLSU). DLSU wants to join ChainCert to start issuing digital credentials to their end users. No fakes, no schemes, just a legitimate university that wants to modernize how they issue diplomas and awards.

Step 1: DLSU navigates to the registration page and fills out the issuer registration form:

Field	**Value**
Organization Name	De La Salle University
Email	dlsu@email.com
Password	pw123
Supporting Documents	CHED Accreditation Certificate, SEC Registration

Step 2: DLSU clicks "Register".

Step 3: The system checks the `accredited_organizations` database.

Step 4: "De La Salle University" IS FOUND in the accredited organizations database. The registration pushes through.

Step 5: The system displays a success message:

Registration Submitted
Your registration has been received and is now pending admin approval. You will be notified via email once your account is approved.

Step 6: The DLSU issuer account is created with status: `PENDING`

Note: At this point, DLSU cannot log in yet. The account requires admin approval first.

Scenario #3: Admin Approves the Issuer Registration

Context: At ChainCert headquarters, the admin, which is the gatekeeper of the system, logs in. They see the Admin Dashboard and click on 'Pending Issuers'. And there's DLSU, waiting for approval. The logic for requiring admin approval is that just because the ORGANIZATION 'De La Salle University' is in the accredited database doesn't mean the PERSON registering is actually from De La Salle University. Think about it, what if some random person just claimed they were from DLSU? They could download a fake certificate, upload it, and say 'I'm from DLSU, let me register.' The accredited organizations check catches ORGANIZATIONS, not whether the person is actually authorized to represent that organization. That's why we need the admin approval step.

The admin is doing a human verification. They're looking at the documents and asking: 'Do these look legitimate? Do they seem real?' They might even contact DLSU separately to verify: 'Hey, did you actually register with us?'

It's that extra layer that prevents posers, people who claim to be from a legitimate organization but aren't actually authorized to represent it. Without admin approval, someone could impersonate DLSU and start issuing fake credentials under DLSU's name. That would destroy DLSU's reputation AND the credibility of our entire system.

Given that they are proven to be legitimate, chaincert will now manually create and deploy a peer node for the aspiring issuer, and only then will the admin click the approve button.

Step 1: The admin logs into the system.

Field	**Value**
Email	admin@email.com
Password	pw123

Step 2: The admin navigates to the Admin Dashboard and clicks on "Pending Issuers".

Step 3: The admin sees DLSU's pending registration:

Organization	**Email**
De La Salle University	dlsu@email.com

Step 4: The admin clicks "View Docs" to review the supporting documents:

- CHED Accreditation Certificate ✓
- SEC Registration ✓
- Verified legitimate organization

Step 5: Behind the Scenes (Pre-Demo Setup – already done offline before approve):

- Create DLSU's Certificate Authority (CA)

```
docker-compose -f compose/compose-ca-dlsu.yaml up -d
ca_dlsu
```

- Register and Enroll DLSU Admin and Peer

```
source registerEnroll.sh createDLSU
```

- Create DLSU's Organization Definition

```
configtxgen -printOrg DLSUMSP > DLSUMSP.json
```

- Fetch, Update, and Submit Channel Configuration
  - Add DLSU to `chaincert-channel` configuration
  - Signed by existing org admins
  - Submit channel update
- Start DLSU Peer and Join Channel

```
docker-compose -f compose/compose-dlsu.yaml up -d
peer0.dlsu.example.com
```

```
peer channel join -b chaincert-channel.block
```

- Install Chaincode on DLSU Peer

```
peer lifecycle chaincode install chaincert.tar.gz
```

Step 6: The admin clicks "Approve".

Step 7: The system performs the following:

- Links the Fabric identity (DLSU admin credentials) to the issuer account
- Changes the account status from `PENDING` to `APPROVED`
- Sends notification email to DLSU

Step 8: The admin sees confirmation:

Issuer Approved

De La Salle University has been approved and can now access the Issuer Dashboard.

Step 9: DLSU can now log in using their registered credentials.

Scenario #4: Issuer Logs In and Creates a Credential Template

Context: It's the beginning of the academic year at DLSU. The registrar's office wants to start issuing digital credentials for their Dean's Lister students — those who maintain a 3.5 GPA or higher. Instead of printing paper certificates that can be lost or forged, they decide to use ChainCert. So DLSU's credential officer logs in with their approved account. But here's the thing, before they can issue ANY credential, they need to define WHAT credential they're issuing. So they create a template.

Step 1: DLSU logs into the system.

Field	**Value**
Email	dlsu@email.com
Password	pw123

Step 2: Login is successful. DLSU is redirected to the Issuer Dashboard.

Step 3: DLSU clicks "Create Template" to create a new credential template.

Step 4: DLSU fills out the credential template form:

Field	**Value**
Template Name	Dean’s Lister Award

Description	Awarded to students who achieved academic excellence with a GPA of 3.5 or higher
Badge Image URL	(optional)
Criteria	Minimum GPA of 3.5, Full-time enrollment
Valid Duration	1 Year

Step 5: DLSU clicks "Create Template".

Step 6: The system:

- Submits the `CreateCredentialTemplate` transaction to the Hyperledger Fabric blockchain
- Records the template in the immutable ledger
- Stores template metadata in DLSU's CouchDB database (Port 9084)

Step 7: Success message displayed:

Template Created Successfully
"Dean's Lister Award" template is now active and ready for issuance.

VERIFICATION – CouchDB (Port 9084)

Navigate to `localhost:9084/_utils` and check the `wallet_issuer_credentials` database.
The new template should be visible.

VERIFICATION – Blockchain Query (Terminal)

```
bash

    # Switch to DLSU admin context
```

```
export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="DLSUMSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrgani
zations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/c
a.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizati
ons/dlsu.example.com/users/Admin@dlsu.example.com/msp

export CORE_PEER_ADDRESS=localhost:12051


# Query credential templates by issuer

peer chaincode query -C chaincert-channel -n chaincert \

  -c
'{"Args":["GetCredentialTemplates","issuer_dlsu_email_com_1
763304108771"]}'
```

Expected Output (sample):

```
json

{

  "templateId": "template_1764095486268_nu3okspsc",

  "name": "Dean's Lister Award",
```

```
  "description": "Awarded to students who achieved
academic excellence...",
  "issuerId": "issuer_dlsu_email_com_1763304108771",
  "status": "active",
  "createdAt": "2024-11-26T10:00:00Z"
}
```

Scenario #5: Issuer Issues a Credential to a User

Context: It's the end of the semester at DLSU. The registrar reviews the grades and identifies all students who made Dean's Lister, which has a GPA 3.5 or higher. Kyle Justin Tan is one of them. He maintained a 3.75 GPA this semester. The DLSU credential officer logs into ChainCert and clicks 'Issue Credential'. They select the 'Dean's Lister Award' template they created earlier, the blueprint is already there.

They search for Kyle by his email: kyle_justin_tan@dlsu.edu.ph and issues the credential to said user

Step 1: DLSU (still logged in) navigates to "Issue Credential".

Step 2: DLSU selects the "Dean's Lister Award" template from the dropdown.

Step 3: DLSU searches for the recipient:

Field	**Value**
Recipient Email	kyle_justin_tan@dlsu.edu.ph

Step 4: The system displays user information:

- Name: Kyle Justin Tan
- Holder ID: 4c7e8e4c20da8359fcdd204e5402d34f

Step 5: DLSU fills in credential details:

Field	**Value**
Academic Year	2024-2025
Term	1st Semester
GPA Achieved	3.75
Issue Date	November 26, 2024
Expiry Date	November 26, 2025

Step 6: DLSU clicks "Issue Credential".

Step 7: The system:

- Generates unique Credential ID:
  `cred_1764095486268_nu3okspsc_4c7e8e4c20da8359fcdd204e5402d34f_1764096024561000000`
- Submits `IssueCredential` transaction to the blockchain
- Records in the shared `chaincert-channel`
- Updates CouchDB (Port 9084) with the issued credential

Step 8: Success message:

Credential Issued Successfully

Dean’s Lister Award has been issued to Kyle Justin Tan.

Credential ID:

`cred_1764095486268_nu3okspsc_4c7e8e4c20da8359fcdd204e5402d34f_1764096024561000000`

Kyle now has a verified, tamper-proof credential on the blockchain. If he ever needs to prove he was a Dean's Lister, he can provide this credential ID or share it with employers or other institutions. They can verify it instantly on the blockchain — and know for certain it's real."

VERIFICATION – Frontend

The issued credential appears in DLSU's "Issued Credentials" list.

VERIFICATION – CouchDB (Port 9084)

Navigate to `localhost:9084/_utils` → `issued_credentials` database.
The new credential document should be visible with all details.

VERIFICATION – Blockchain Query (Terminal)

bash

```
# Query issued credentials by DLSU

peer chaincode query -C chaincert-channel -n chaincert \

  -c
'{"Args":["GetIssuedCredentials","issuer_dlsu_email_com_176
3304108771"]}'


# Query credentials received by Kyle (holder)

peer chaincode query -C chaincert-channel -n chaincert \
```

```
  -c
'{"Args":["GetCredentialsByHolder","4c7e8e4c20da8359fcdd204
e5402d34f"]}'


# Query specific credential by ID
peer chaincode query -C chaincert-channel -n chaincert \
  -c
'{"Args":["GetCredential","cred_1764095486268_nu3okspsc_4c7
e8e4c20da8359fcdd204e5402d34f_1764096024561000000"]}'
```

Expected Output (sample):

json

```
{
  "credentialId":
"cred_1764095486268_nu3okspsc_4c7e8e4c20da8359fcdd204e5402d
34f_1764096024561000000",
  "templateId": "template_1764095486268_nu3okspsc",
  "templateName": "Dean's Lister Award",
  "holderId": "4c7e8e4c20da8359fcdd204e5402d34f",
  "holderName": "Kyle Justin Tan",
  "issuerId": "issuer_dlsu_email_com_1763304108771",
```

```
  "issuerName": "De La Salle University",
  "status": "active",
  "issuedAt": "2024-11-26T10:15:00Z",
  "expiresAt": "2025-11-26T10:15:00Z"
}
```

Scenario #6: Employer Verifies a Submitted Credential

Context: A few months later, Kyle is applying for a job. In his application, he proudly includes that he was a Dean's Lister at DLSU. The employer doesn't want to rely on screenshots or PDFs that can be edited, so their IT or HR team uses ChainCert to verify the validity of said credential.

Ways to Verify a Credential (Current & Planned)

Before we dive into the demo, let's clarify the different ways a verifier can verify credentials in ChainCert, some are implemented now, others are planned for the future:

1. Verify by Credential ID (Currently Implemented — CLI only)

The user (Kyle) sends the specific credential ID to the verifier. The verifier enters this ID into the verification system.

Current Status: This works via blockchain query (terminal/CLI) — you query the chaincode directly with the credential ID.

Planned: A web-based Verifier Dashboard where you paste the credential ID, and it shows the status (active, revoked, expired), issuer name, holder name, and all credential details in a user-friendly interface.

2. Verify via Profile Link (Planned — Not Yet Implemented)

The user shares a direct link to their ChainCert profile (e.g., https://chaincert.io/profile/4c7e8e4c20da8359fcdd204e5402d34f).

When the verifier clicks this link, they see a profile preview page showing:

- The user's name and profile info
- A credential wallet/badge display — all credentials the user has set to public
- The validity status of each credential (active, revoked, expired)
- Metadata (issuer, issue date, expiry date, credential type, etc.)

Key Point: The user controls visibility. If a credential is marked private, it won't show up on this public profile page. Only credentials marked public are visible to verifiers who visit the profile link.

3. Verify by Searching the User's Name (Planned — Not Yet Implemented)

The verifier goes to the Verifier Dashboard and searches for the user by name (e.g., "Kyle Justin Tan").

If the user has set their profile to public, the search will return their profile, and the verifier can click through to see the same profile preview page as in Method 2 — credential wallet, validity status, metadata, etc.

Key Point: This only works if the user has made their profile publicly searchable. If the profile is private, the verifier won't find them in the search.

4. Verify via QR Code (Planned — Not Yet Implemented)

The user generates a QR code for a specific credential or their entire profile. The verifier scans the QR code with a mobile device, and it redirects them to either:

- The credential verification page (for a single credential), or
- The user's profile preview page (for their full credential wallet)

This is useful for in-person verification scenarios — job interviews, document checks at offices, etc.

Why These Verification Methods Matter

The Old Problem:

Before ChainCert, employers had to manually contact institutions to verify credentials. This took days or weeks, and there was no guarantee the document was real. Forgeries are common.

The Solution:

With ChainCert, given these list of ways to verify, verifiers can verify instantly. Job offers that would normally be delayed for weeks while waiting for verification can now happen in days. Students and professionals get instant proof of their accomplishments, no bureaucracy, no doubt

VERIFICATION METHOD – Blockchain Query (Terminal)

bash

```
# Verify credential by ID

peer chaincode query -C chaincert-channel -n chaincert \

  -c
'{"Args":["VerifyCredential","cred_1764095486268_nu3okspsc_
4c7e8e4c20da8359fcdd204e5402d34f_1764096024561000000"]}'
```

Expected Output (Valid Credential):

json

```
{

  "valid": true,

  "credentialId":
"cred_1764095486268_nu3okspsc_4c7e8e4c20da8359fcdd204e5402d
34f_1764096024561000000",

  "status": "active",

  "holderName": "Kyle Justin Tan",

  "issuerName": "De La Salle University",

  "templateName": "Dean's Lister Award",

  "issuedAt": "2024-11-26T10:15:00Z",
```

```
  "expiresAt": "2025-11-26T10:15:00Z",
  "blockchainVerified": true,
  "message": "Credential is valid and has not been revoked."
}
```

Expected Output (Revoked Credential – when revocation is used):

json

```
{
  "valid": false,
  "credentialId": "cred_...",
  "status": "revoked",
  "message": "Credential has been revoked.",
  "revokedAt": "2024-11-27T08:00:00Z",
  "revocationReason": "Academic integrity violation"
}
```

Get Credential History (Audit Trail)

bash

```
peer chaincode query -C chaincert-channel -n chaincert \
```

```
  -c
'{"Args":["GetCredentialHistory","cred_1764095486268_nu3oks
psc_4c7e8e4c20da8359fcdd204e5402d34f_1764096024561000000"]}
'
```

Expected Output (sample):

```
json
[
  {
    "txId": "abc123...",
    "timestamp": "2024-11-26T10:15:00Z",
    "action": "ISSUED",
    "actor": "De La Salle University"
  }
]
```

Added / Planned Scenarios (Not Yet Implemented or Partially Implemented)

These are intended flows to show the whole picture of ChainCert. Some have partial backend/chaincode support, but no full UI; others are still design-level only.

Scenario #8: User Dashboard – Accept/Decline & Privacy Controls (Planned)

Context: Kyle has now received multiple credentials from DLSU and other institutions. However, all credentials submitted to him are subject to the receivers approval. This is because without approval, credential issuance becomes automatic, filling Kyle's wallet with unwanted credentials—disciplinary records, personal data, medical information—that he never consented to share. This violates privacy laws (GDPR/CCPA) and destroys his reputation when verifiers see everything. Approval-first ensures Kyle controls his identity, not the issuer. He accepts only relevant credentials, keeps sensitive ones private, maintains a clean wallet, and complies with data protection laws. It's the difference between an employer automatically posting all your performance reviews online versus you choosing which accomplishments to highlight on your resume. In ChainCert, credentials remain pending until Kyle explicitly accepts them; only then are they added to the blockchain and wallet. This respects self-sovereign identity principles—Kyle owns his credentials, not DLSU. he wants full control over which credentials are visible to whom. To conclude, the ChainCert system allows him to accept, decline, or manage privacy settings for each credential directly from his dashboard.

Intended Flow (UI NOT IMPLEMENTED, LOGIC PARTLY PLANNED):

Step 1: User logs in and opens "My Credentials".

Step 2: For each credential, user sees:

- Status: Pending Acceptance / Accepted / Declined
- Visibility: Public / Private

Step 3: When a new credential is issued:

- It appears as "Pending Acceptance".
- User can click "Accept" or "Decline".

Step 4: After acceptance:

- Credential becomes "Accepted".
- User can toggle "Public" or "Private".
    - Public: visible on shareable profile and searchable to verifiers.
    - Private: only visible to the user (still verifiable by ID if holder shares direct ID).

Step 5 (Planned):

- "Share Profile" button generates a URL like: `https://chaincert.io/profile/4c7e8e4c20da8359fcdd204e5402d34f`
- Only public credentials appear on this profile.

Step 6 (Planned):

- "Export Transcript" generates a PDF listing all selected credentials (like transcript of records or awards).

Status:

- Blockchain/CouchDB base data: already there (credentials per holder).
- Acceptance / Decline flags, visibility flags: NOT YET IMPLEMENTED.

- Profile sharing and transcript export: NOT YET IMPLEMENTED.

Scenario #9: Credential Revocation – Issuer UI (Partially Implemented)

Context: Three weeks ago, DLSU issued a Bachelor of Science in Computer Science degree credential to Juan Dela Cruz (User ID: user_juan_2024). The credential was recorded on the blockchain and Juan was thrilled that he even shared his credential QR code on his social medias.

However, this morning, the Academic Integrity Office discovered that Juan submitted plagiarized coursework during his final year. After reviewing the evidence, DLSU's Academic Director decided that Juan's degree must be revoked immediately.

Current Backend/Chaincode Status:

- `RevokeCredential()` is present in chaincode design and/or partially wired, but full issuer UI + end-to-end test is not completed.

Intended UI Flow:

Step 1: Issuer logs into Issuer Dashboard → "Issued Credentials".

Step 2: For a specific credential (e.g., Kyle's Dean's Lister award), issuer clicks: "View / Manage".

Step 3: Issuer clicks "Revoke Credential", enters:

- Revocation Reason: "Academic integrity violation"
- Optional internal notes

Step 4: System calls `RevokeCredential` chaincode function:

- Changes on-chain status from `active` to `revoked`
- Writes `revokedAt` and `revocationReason`
- Updates CouchDB document as well

Step 5: User notification:

- User dashboard shows credential as "Revoked" with reason.
- Any subsequent verification shows it as revoked (this is already represented in `VerifyCredential` expected output).

Status:

- Chaincode logic: partially or conceptually implemented
- UI for revocation and user-facing messages: NOT FULLY IMPLEMENTED / NOT DEMO-READY

Scenario #10: Batch Issuance (Planned)

Context: DLSU wants to issue Dean's Lister credentials to hundreds of students at once.

Intended Flow (NO IMPLEMENTATION YET):

Step 1: Issuer in Dashboard clicks "Batch Issue".

Step 2: Uploads CSV with columns:

- email
- fullName
- academicYear
- term
- GPA / criteria fields

Step 3: Backend validates:

- All emails correspond to registered users (or auto-create, depending on policy).

Step 4: For each row:

- Calls `IssueCredential` chaincode function
- Writes documents into CouchDB

Step 5: Summary shows:

- Total attempted
- Successful issues
- Failed (with error reasons)

Status:

- Purely planned. No existing chaincode or UI in this implementation.

Scenario #11: Admin Analytics and System-wide Audit Logs (Planned)

Context: Admin wants visibility over how the system is being used.

Intended Capabilities (NOT IMPLEMENTED):

- View total number of:
    - Registered, pending, and revoked issuers
    - Other necessary details that an admin needs to know
- Filter by date range, institution, credential type.
- View suspicious patterns:
    - Many revocations from the same issuer

- Unusual verification bursts

Implementation Status:

- On-chain data and CouchDB documents make this possible, but
- No analytics pipeline or dashboard UI implemented in current build.

CouchDB Database Reference

_id	email	password (hashed)	role	_id
4c7e8e4c20da8359fcdd204e5402d34f	kyle_justin_tan@dlsu.edu.ph	$2b$10$...	user	4c7e8e4c20da8359fcdd204e5402d34f
4c7e8e4c20da8359fcdd204e540347b0	mema@email.com	$2b$10$...	user	4c7e8e4c20da8359fcdd204e540347b0
a8fc2c2ad56fc76be0b2e716b80002a6	admin@email.com	$2b$10$...	admin	a8fc2c2ad56fc76be0b2e716b80002a6

Issuer Credential Database (Port 9084 – DLSU)

Database	Description
issued_credentials	Stores all credentials issued by DLSU
wallet_issuer_credentials	Stores DLSU's issuer identity/wallet info

---

Terminal Commands Quick Reference

Switch to DLSU Context

bash

```
export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="DLSUMSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrgani
zations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/c
a.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizati
ons/dlsu.example.com/users/Admin@dlsu.example.com/msp

export CORE_PEER_ADDRESS=localhost:12051
```

Query Credential Templates

bash

```
peer chaincode query -C chaincert-channel -n chaincert \

  -c '{"Args":["GetCredentialTemplates","<issuer_id>"]}'
```

Query Issued Credentials by Issuer

bash

```
peer chaincode query -C chaincert-channel -n chaincert \
  -c '{"Args":["GetIssuedCredentials","<issuer_id>"]}'
```

Query Credentials by Holder

bash

```
peer chaincode query -C chaincert-channel -n chaincert \
  -c '{"Args":["GetCredentialsByHolder","<holder_id>"]}'
```

Verify a Credential

bash

```
peer chaincode query -C chaincert-channel -n chaincert \
  -c '{"Args":["VerifyCredential","redential_id>"]}'
```

Get Credential History

bash

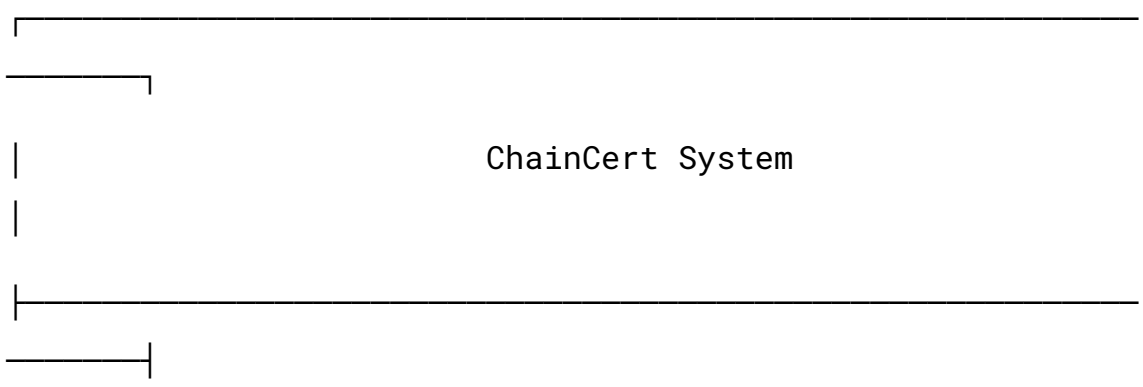
```
peer chaincode query -C chaincert-channel -n chaincert \
  -c '{"Args":["GetCredentialHistory","redential_id>"]}'
```

Demo Flow Summary

1. Show Failed Registration (Non-accredited org) → Error message
2. Show Successful Registration (DLSU – accredited) → Pending status
3. Admin Login → View pending issuers → (Node setup already done) → Click Approve
4. DLSU Login → Successful login to Issuer Dashboard
5. Create Credential Template → Show Frontend + CouchDB + Blockchain Query
6. Issue Credential to User → Show Frontend + CouchDB + Blockchain Query
7. Verify Credential → Blockchain Query (terminal only)
8. Explain Future/Planned Flows (no UI yet):
    - Verifier Dashboard
    - User acceptance / privacy / profile sharing / transcript export
    - Issuer revocation UI & batch issuance
    - Admin analytics and system-wide logs

System Architecture Overview

text



```
|  Frontend (React)
|
|  ├── Login/Registration
|
|  ├── Admin Dashboard (Approve Issuers)
|
|  ├── Issuer Dashboard (Create Templates, Issue
Credentials)     |
|  ├── User Dashboard (View Credentials) [Partial]
|
|  └── Verifier Dashboard (Web-based Verification)
[PLANNED]      |
├──────────────────────────────────────────────────────
──────┤
|  Backend API (Node.js/Express) - Port 4000
|
|  ├── sdk-server-fixed.js (Fabric Gateway Connection)
|
|  ├── authController.js (Authentication & Registration)
|
|  ├── issuerCredentialController.js (Credential
Operations)      |
```

```
|   └── couchDB_service.js (Off-chain Data Storage)
|
├──────────────────────────────────────────────────────────
──────┤
|  Hyperledger Fabric Network
|
|   ├── Org1MSP (ChainCert Org 1) - Port 7051
|
|   ├── Org2MSP (ChainCert Org 2) - Port 9051
|
|   ├── DLSUMSP (De La Salle University) - Port 12051
|
|   ├── Orderer - Port 7050
|
|   └── chaincert-channel (Shared Ledger)
|
├──────────────────────────────────────────────────────────
──────┤
|  Chaincode (index.js)
|
|   ├── CreateCredentialTemplate()
|
```

```
|   ├── IssueCredential()
|
|   ├── RevokeCredential()     [PARTIAL / BACKEND-LEVEL]
|
|   ├── VerifyCredential()
|
|   ├── GetCredentialsByHolder()
|
|   ├── GetCredentialHistory()
|
|   └── GetIssuedCredentials()
|
├──────────────────────────────────────────────────────────
──────┤
|  Data Storage
|
|   ├── Blockchain Ledger (Immutable Transaction Records)
|
|   ├── CouchDB Port 5985 (User Authentication - Main App)
|
|   └── CouchDB Port 9084 (DLSU Issuer Credential Storage)
|
```

End of Walkthrough Document

This version includes both the implemented demo scenarios and the planned but not yet implemented flows, so panelists can clearly see the current working features and the full envisioned picture of how ChainCert is meant to operate end-to-end.